

Design a Secure Architecture for an Electric Vehicle Charging Management System with Secure Communication and Billing

1. Introduction

Electric vehicle charging infrastructure has become a critical part of modern smart transportation. An EV charging management system connects drivers, charging stations, cloud services, payment gateways, and utility providers. Because these components exchange financial, operational, and personal information over public networks, cybersecurity is a primary requirement. DevSecOps integrates security into every phase of software development and operations so that vulnerabilities are identified early, deployments remain secure, and systems are continuously monitored. This report explains a secure architecture for an EV charging management system by combining secure communication protocols, identity management, secure billing, continuous monitoring, and automated security testing.

2. Problem Statement

Many EV charging platforms rely on cloud connectivity and exposed APIs. If authentication is weak or communication is not encrypted, attackers can manipulate charging sessions, steal payment information, impersonate charging stations, or disrupt service through denial-of-service attacks. Traditional development often adds security only after implementation, resulting in higher costs and increased risk. DevSecOps addresses these issues by embedding security practices into planning, coding, testing, deployment, and monitoring.

3. Objectives

The primary objective is to design a secure and scalable architecture that protects users, charging stations, and backend services. The solution aims to ensure confidentiality, integrity, and availability of data, provide secure billing and payment processing, enforce strong authentication and

authorization, automate vulnerability detection in the CI/CD pipeline, and maintain compliance with industry standards such as PCI DSS and ISO 27001.

4. Secure Architecture Overview

The architecture follows a Zero Trust approach. Every communication channel is authenticated and encrypted using TLS 1.3, while mutual TLS authenticates charging stations. API Gateway validates requests before forwarding them to backend services. OAuth 2.0 and OpenID Connect manage user authentication, while role-based access control restricts administrative functions. Secrets are stored in a secure vault instead of source code. Runtime logs are collected by a SIEM platform for real-time threat detection and incident response. Billing services interact securely with external payment providers using tokenized payment information.

5. Proposed Secure Architecture Diagram

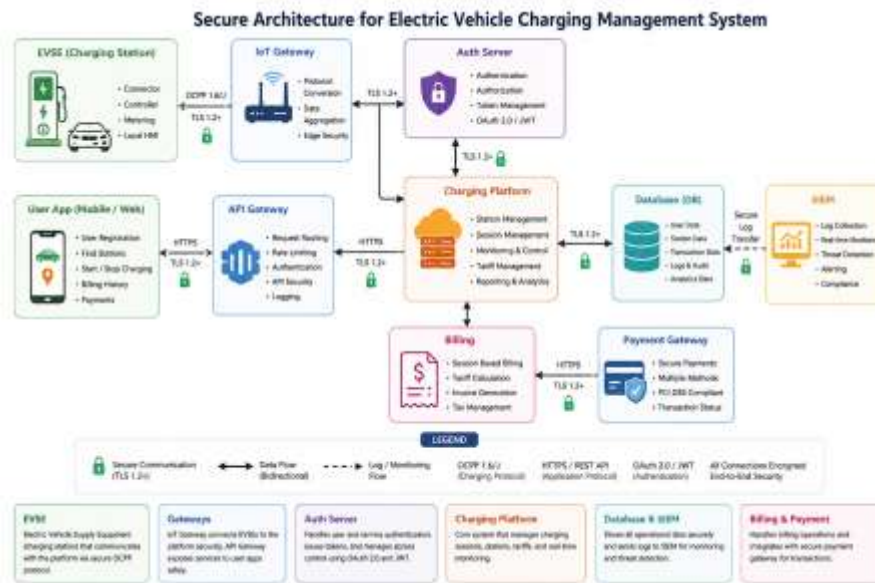


Figure 1: illustrates a high-level secure architecture. All communication channels are protected using TLS 1.3, charging stations authenticate through mutual TLS, and operational logs are continuously forwarded to the SIEM platform.

6. Major Components

Component	Role	Security Mechanism
Mobile App	Starts charging and payments	OAuth 2.0, MFA
API Gateway	Validates requests	JWT, Rate limiting, WAF
Charging Platform	Session management	RBAC, Logging
Billing Service	Calculates cost	PCI DSS, Tokenization
Database	Stores users and transactions	AES-256 Encryption
SIEM	Threat monitoring	Centralized Logs

7. Secure Communication

Secure communication is established using TLS 1.3 for all external APIs and mutual TLS between charging stations and backend services. Digital certificates issued through a public key infrastructure prevent unauthorized devices from joining the network. JSON Web Tokens provide secure session management, while API requests are validated through an API gateway. Sensitive information stored in databases is encrypted using AES-256 and encryption keys are managed through a centralized key management system.

8. Secure Billing

Billing security is essential because financial information is processed continuously. Instead of storing payment card details, tokenization replaces sensitive information with secure tokens. Payment processing follows PCI DSS requirements and every transaction is digitally signed to prevent tampering. Immutable audit logs support financial investigations and fraud detection algorithms identify abnormal charging behavior such as repeated failed transactions or impossible charging locations.

9. DevSecOps Implementation

The CI/CD pipeline begins with source code management in Git. Every commit triggers static application security testing, dependency scanning, secret detection, unit testing, container image scanning, and dynamic

application security testing. Only builds that satisfy security policies are deployed to Kubernetes. Infrastructure is managed using Infrastructure as Code and runtime monitoring with Prometheus, Grafana, and SIEM ensures continuous visibility into system health and attacks.

10. Risk Analysis

The highest risks include credential theft, payment fraud, API abuse, ransomware, and denial-of-service attacks. Risk is reduced through multi-factor authentication, least privilege access, encrypted communication, continuous patch management, endpoint protection, and automated incident response playbooks. Security metrics and CVSS scores help prioritize remediation.

11. Compliance and Best Practices

The architecture aligns with ISO/IEC 27001 for information security management, PCI DSS for payment processing, the NIST Cybersecurity Framework for risk management, and the OWASP Top 10 and ASVS for application security. Secure coding standards, regular penetration testing, backup validation, disaster recovery drills, and periodic security awareness training improve organizational resilience.

12. Conclusion

A secure EV charging management system requires more than encrypted communication. It requires security throughout the entire software lifecycle. By adopting DevSecOps, organizations automate vulnerability detection, integrate security testing into CI/CD, protect payment information, monitor infrastructure continuously, and respond quickly to cyber threats. This architecture improves customer trust, regulatory compliance, operational reliability, and long-term scalability for smart transportation ecosystems.

13. Threats and Mitigations

Threat	Impact	Mitigation
MITM	Data theft	TLS 1.3 + mTLS

SQL Injection	Database compromise	Parameterized queries
DDoS	Service outage	WAF + Rate limiting
Credential Theft	Unauthorized access	MFA + RBAC
Ransomware	Data loss	Backups + EDR

14. References

- NIST Cybersecurity Framework
- PCI DSS v4.0
- OWASP Top 10
- OWASP ASVS
- ISO/IEC 27001